# OZ Lab

est. August 2024

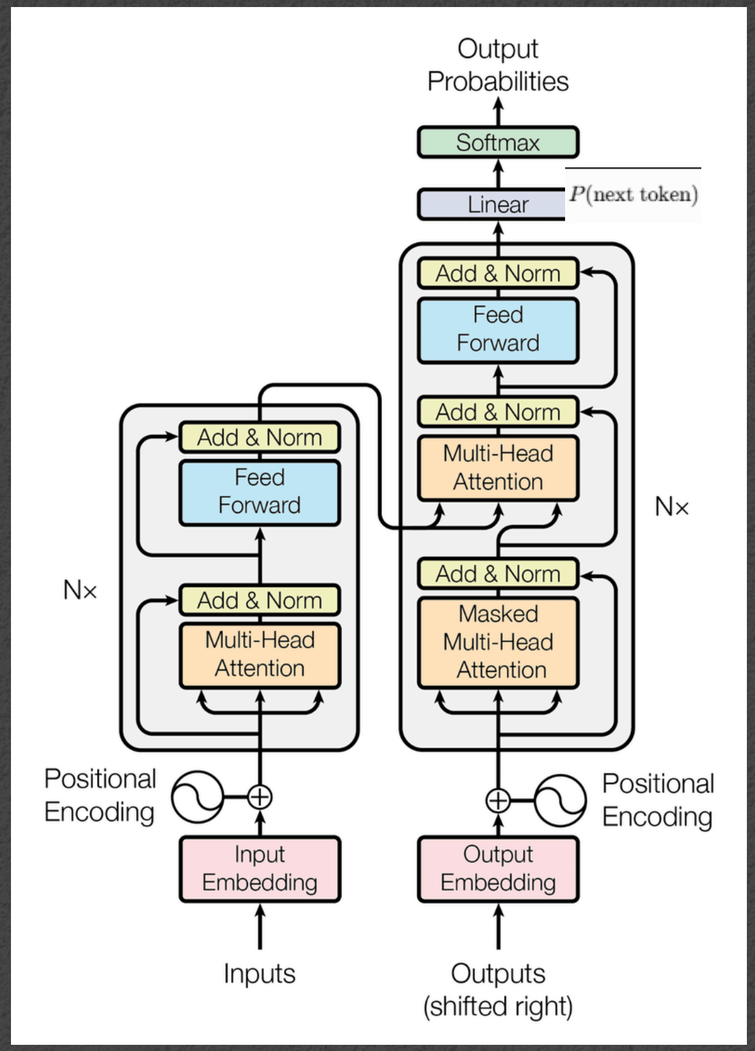
## #1 - 목차 및 머신러닝 개요

@oz.lab__



@oz.lab__

# Intro to Transformers




$$\text{Token} \xrightarrow{\text{Embedding}} X \xrightarrow[\text{weighted sum of } V]{\text{softmax}(QK^\top/\sqrt{d})} X' \xrightarrow{\text{MLP}} X'' \xrightarrow{\text{repeat } L \text{ times}} H_L \xrightarrow{W_U} \text{logits} \xrightarrow{\text{softmax}} P(\text{next token})$$

## 파트 1

1. 벡터
2. 행렬
3. Linear Transformation
4. 뉴럴 네트워크
5. 손실
6. Gradient
7. Backprop

## 파트 2

1. 엠베딩
2. Q, K V
3. (QK)^T
4. Softmax
5. Attention
6. MLP
7. Residual
8. LayerNorm

@oz.lab__

# Intro to Transformers

Vaswani, A., Shazeer, N., et al. “Attention Is All You Need,” in Advances in Neural Information Processing Systems (NeurIPS), 2017.


**Abstract**

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.


Li, Naihan, et al. “Neural Speech Synthesis with Transformer N


**Abstract**

Although end-to-end neural text-to-speech (TTS) methods (such as Tacotron2) are proposed and achieve state-of-the-art performance, they still suffer from two problems: 1) low efficiency during training and inference; 2) hard to model long dependency using current recurrent neural networks (RNNs). Inspired by the success of Transformer network in neural machine translation (NMT), in this paper, we introduce and adapt the multi-head attention mechanism to replace the RNN structures and also the original attention mechanism in Tacotron2. With the help of multi-head self-attention, the hidden states in the encoder and decoder are constructed in parallel, which improves training efficiency. Meanwhile, any two inputs at different times are connected directly by a self-attention mechanism, which solves the long range dependency problem effectively. Using phoneme sequences as input, our Transformer TTS network generates mel spectrograms, followed by a WaveNet vocoder to output the final audio results. Experiments are conducted to test the efficiency and performance of our new network. For the efficiency, our Transformer TTS network can speed up the training about 4.25 times faster compared with Tacotron2. For the performance, rigorous human tests show that our proposed model achieves state-of-the-art performance (outperforms Tacotron2 with a gap of 0.048) and is very close to human quality (4.39 vs 4.44 in MOS).

# Intro to Transformers

Transformer 구조가 트이게 된 계기

1. 코드 동작 이해 후

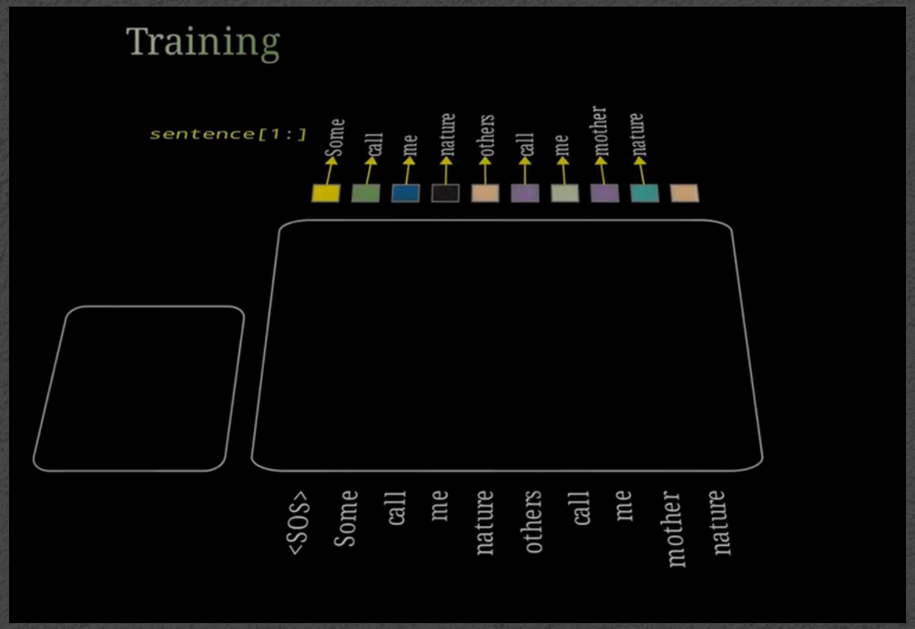
2. 동작의 시각적 이해

```python
class MHAttn(nn.Module):

    def forward(self, q, k, v, causal=False):
        q = self.w_q(q) # B, L, D
        k = self.w_k(k)
        v = self.w_v(v)
        breakpoint()
        q = q.view(
            q.shape[0], q.shape[1], self.n_heads, q.shape[-1] // self.n_heads
        ).transpose(1, 2) # B, 8, L, D // 8
        k = k.view(
            k.shape[0], k.shape[1], self.n_heads, k.shape[-1] // self.n_heads
        ).transpose(1, 2)
        v = v.view(
            v.shape[0], v.shape[1], self.n_heads, v.shape[-1] // self.n_heads
        ).transpose(1, 2)

        attn = torch.matmul(q, k.transpose(-2, -1)) / (q.shape[-1] ** 0.5)
        if causal:
            mask = torch.triu(torch.ones(attn.shape[-2], attn.shape[-1]), 1).to(
                attn.device
            )
            attn = attn.masked_fill(mask == 1, -1e9)
        attn = torch.softmax(attn, dim=-1)
        out = torch.matmul(attn, v) # B, 8, Lq, D // 8
```

Attention을 통해 "특정 단어에 주목"한다



# Intro to Transformers

1. 벡터와 행렬이 무엇인지?

2. 행렬곱, 벡터 내적이란? 입력과 출력은?

3. 고등수학 개념: exponential, mean, std 등

# Intro to Transformers (숙제)..

**선형대수학을 시각적으로 이해해야 하는 이유**
https://www.youtube.com/watch?v=ic_hG2M2nG0&list=PLkoaXOTFHiqhVDo0nWybNmihCP_4BjOFR

**벡터란 무엇인가?**
https://www.youtube.com/watch?v=ArgTeYVuJUo&list=PLkoaXOTFHiqhVDo0nWybNmihCP_4BjOFR&index=2

**선형결합, 생성, 기저 벡터**
https://www.youtube.com/watch?v=2CcCOgDilO8&list=PLkoaXOTFHiqhVDo0nWybNmihCP_4BjOFR&index=3

**선형변환과 행렬**
https://www.youtube.com/watch?v=35ESC-g49fY&list=PLkoaXOTFHiqhVDo0nWybNmihCP_4BjOFR&index=4

**선형변환의 합성과 행렬의 곱셈**
https://www.youtube.com/watch?v=gXjCxYuEybQ&list=PLkoaXOTFHiqhVDo0nWybNmihCP_4BjOFR&index=5

**내적과 쌍대성**
https://www.youtube.com/watch?v=zSd7jd2HGhM&list=PLkoaXOTFHiqhVDo0nWybNmihCP_4BjOFR&index=10

0. Intro to Math for ML (3B1B) [HW]
    1. Intro to ML (Stanford) 매주 1-2시간
    2. Intro to DL (3B1B) - 매주 2시간
        a. But what is a NN?
        b. Gradient Descent
        c. Analyzing NN
        d. Backpropagation
        e. LLM
        f. Transformers
        g. Attention
        h. Memory
        i. Cross Entropy
        j. Diffusion / CLIP (Optional)
    3. Mechanistic Interpretability
        a. Mathematical Framework for Transformer Circuits
    4. Hands on Mech Interp (Do your own)

ML 슬라이드 :
https://docs.google.com/presentation/d/1UQmTComnPEBzHNvITBWBa4iKPUHCRxiFNsufahSHrKE/edit?slide=id.p2#slide=id.p2

ML 랩:
https://colab.research.google.com/drive/1q8_VBIU4qridFjtp2m_umrVLnFsGKk1e#scrollTo=w2l02c005